

Практическая работа 6

ASP.NET Core (.NET 10)

Тема: конфигурация приложения, провайдеры конфигурации, объединение источников, проекция на классы и IOptions

Сквозное задание: "CampusHub.ConfigCenter"

Учебный сервис диагностики и управления конфигурацией внутреннего портала колледжа

Работа должна быть выполнена как цельное ТЗ для одного веб-приложения. Разрешается небольшое изменение названий файлов и классов, но логика задания, обязательные маршруты и проверяемые темы должны быть сохранены.

Формат сдачи: проект + отчет + скриншоты ключевых проверок

Паспорт работы

Формат	Одна интегрированная практическая работа вместо набора разрозненных упражнений.
Цель	Научиться собирать конфигурацию ASP.NET Core из нескольких источников, анализировать секции, создавать собственный провайдер конфигурации, связывать данные конфигурации с классами и передавать их через IOptions.
Средства	.NET 10 SDK, шаблон Empty Web App (dotnet new web), Visual Studio / Rider / VS Code, браузер или Postman.
Результат	Рабочее веб-приложение CampusHub.ConfigCenter, набор конфигурационных файлов, собственный провайдер конфигурации и краткий отчет с пояснением приоритета источников.

1. Что должен освоить студент

- понимать роль IConfiguration и уметь получать данные конфигурации через индексатор, GetSection() и Dependency Injection;
- подключать нефайловые провайдеры конфигурации: аргументы командной строки, переменные среды и in-memory коллекцию;
- подключать файловые провайдеры: JSON, XML и INI;
- объяснять, как объединяются несколько источников конфигурации и почему при конфликте одинаковых ключей побеждает значение из более позднего источника;
- анализировать конфигурацию через GetChildren(), GetConnectionString() и список провайдеров IConfigurationRoot;
- создавать собственный провайдер конфигурации на базе класса ConfigurationProvider;
- выполнять проекцию конфигурации на простые и сложные классы через Bind() и Get<T>();
- регистрировать настройки через IOptions<T> и использовать их в endpoint и в custom middleware.

2. Сквозной сценарий задания

Необходимо создать веб-приложение CampusHub.ConfigCenter, которое играет роль учебного центра диагностики конфигурации внутреннего портала колледжа. Приложение должно показывать итоговые настройки портала, демонстрировать объединение нескольких источников конфигурации, выводить структуру отдельных секций, показывать строки подключения, использовать собственный провайдер конфигурации и передавать настройки в middleware через механизм Options.

3. Какие темы и где именно отрабатываются

Тема	Что реализует студент	Как проверить
------	-----------------------	---------------

IConfiguration и основы конфигурации	Получение значений через индекатор и DI на маршруте /config/raw.	Ответ содержит несколько ключей из разных источников.
Нефайловые источники	Использование environment variables, commandLineArgs и AddInMemoryCollection().	Маршрут /config/effective показывает итоговые значения.
JSON / XML / INI	Подключение appsettings.json, appsettings.Development.json, portal.xml и notifications.ini.	В ответах видны значения из файловых источников.
Объединение конфигураций	Повторение одинаковых ключей в нескольких источниках и демонстрация итогового значения.	Студент объясняет, какой источник победил и почему.
Анализ конфигурации	GetSection(), GetChildren(), GetConnectionString(), IConfigurationRoot.Providers.	Маршруты /config/tree, /config/connection, /config/providers.
Собственный провайдер	TextConfigurationProvider + TextConfigurationSource + extension method AddTextFile().	Маршрут /config/custom читает данные из customsettings.txt.
Проекция на классы	Bind() или Get<PortalOptions>() для сложного объекта PortalOptions.	Маршрут /config/bind возвращает связанный объект.
IOptions	Configure<PortalOptions>() и Configure<NotificationOptions>(); использование в endpoint и middleware.	Маршрут /config/options и заголовки ответа X-Portal-Title / X-Portal-Semester.

4. Функциональные требования к приложению

1. Создать проект шаблона Empty Web App под .NET 10.
2. Подготовить минимум следующие файлы конфигурации: appsettings.json, appsettings.Development.json, portal.xml, notifications.ini, customsettings.txt и Properties/launchSettings.json.
3. В appsettings.json определить секции Portal, ConnectionStrings и FeatureFlags. В секции Portal обязательно должны быть вложенный объект Admin и массив Modules.
4. В appsettings.Development.json переопределить хотя бы два значения из appsettings.json.
5. В launchSettings.json для выбранного профиля указать как минимум одну environment variable и один набор commandLineArgs, влияющих на конфигурацию приложения.
6. Подключить XML-файл через AddXmlFile(...), INI-файл через AddIniFile(...), а in-memory настройки — через AddInMemoryCollection(...).
7. Реализовать собственный провайдер конфигурации TextConfigurationProvider, источник TextConfigurationSource и метод расширения AddTextFile(...).
8. Настроить порядок подключения источников осмысленно и продемонстрировать минимум два конфликта одинаковых ключей между источниками.

9. Создать классы PortalOptions, AdminOptions и NotificationOptions. Для PortalOptions использовать вложенный объект и коллекцию, чтобы продемонстрировать привязку сложной структуры.
10. На маршруте /config/bind показать результат привязки конфигурации к объекту через Bind() или Get<T>().
11. Зарегистрировать настройки через IOption<T> и на маршруте /config/options вернуть значения из IOption<PortalOptions> и IOption<NotificationOptions>.
12. Создать custom middleware PortalHeaderMiddleware, который получает IOption<PortalOptions> и добавляет в ответ заголовки X-Portal-Title и X-Portal-Semester.
13. Реализовать маршрут /config/providers, который выводит список реально подключенных провайдеров конфигурации из IConfigurationRoot.
14. Реализовать маршрут /config/tree, который рекурсивно выводит содержимое выбранной секции через GetChildren().
15. Реализовать маршрут /config/connection, который получает строку подключения через GetConnectionString("DefaultConnection").

5. Обязательные файлы и ключи конфигурации

Файл	Назначение	Что должно лежать внутри	Примечание
appsettings.json	Базовая конфигурация	Portal:Title, Portal:Semester, Portal:Admin:Name, Portal:Admin:Email, Portal:Modules, ConnectionStrings:DefaultConnection, FeatureFlags:*	Обязателен
appsettings.Development.json	Переопределение для Development	Минимум два ключа, совпадающих с appsettings.json	Обязателен
portal.xml	Дополнительные настройки портала	Например: Portal:CampusName, Portal:Dean или Portal:Building	Обязателен
notifications.ini	Настройки уведомлений	Секция Notifications с Sender, Channel, Signature	Обязателен
customsettings.txt	Файл для собственного провайдера	Данные в формате: строка с ключом, следующая строка — значение	Обязателен
launchSettings.json	Параметры запуска	ASPNETCORE_ENVIRONMENT, как минимум одна env var и commandLineArgs	Обязателен

Рекомендуемый проверочный набор конфликтов ключей: Portal:Title должен существовать в appsettings.json, appsettings.Development.json и commandLineArgs; Portal:SupportEmail — в appsettings.json и environmentVariables; Notifications:Sender — в notifications.ini и in-memory коллекции. На маршруте /config/effective студент должен показать финальные значения этих ключей.

6. Обязательные маршруты и ожидаемое поведение

Маршрут	Ожидаемое поведение	Какая тема проверяется
/	Стартовая страница с кратким описанием проекта и ссылками на тестовые маршруты.	Терминальный endpoint
/config/raw	Возвращает несколько значений через IConfiguration: например Portal:Title, Portal:SupportEmail, Notifications:Sender.	Индексатор IConfiguration и DI
/config/section/portal	Показывает данные секции Portal через GetSection().	GetSection
/config/tree	Рекурсивно выводит структуру секции Portal или другой выбранной секции.	GetChildren и анализ конфигурации
/config/connection	Возвращает значение GetConnectionString("DefaultConnection").	GetConnectionString
/config/providers	Показывает список провайдеров конфигурации из IConfigurationRoot.Providers.	Анализ провайдеров
/config/custom	Возвращает значения, считанные собственным провайдером из customsettings.txt.	Custom provider
/config/bind	Возвращает объект PortalOptions, полученный через Bind() или Get<PortalOptions>().	Проекция на классы
/config/options	Возвращает значения из IOptions<PortalOptions> и IOptions<NotificationOptions>.	IOptions
/config/effective	Показывает итоговые значения конфликтующих ключей и текущее окружение.	Объединение источников и приоритеты

7. Минимальный набор классов и рекомендуемая структура проекта

Рекомендуемая структура проекта (названия можно слегка менять, но смысл должен сохраниться):

```
CampusHub.ConfigCenter/  
├─ Program.cs  
├─ Configuration/  
│   ├── TextConfigurationProvider.cs  
│   ├── TextConfigurationSource.cs  
│   └─ TextConfigurationExtensions.cs  
├─ Middleware/  
│   └─ PortalHeaderMiddleware.cs  
├─ Models/  
│   ├── PortalOptions.cs  
│   ├── AdminOptions.cs  
│   └─ NotificationOptions.cs  
├─ Properties/launchSettings.json  
├─ appsettings.json  
└─ appsettings.Development.json
```

- |─ portal.xml
- |─ notifications.ini
- └─ customsettings.txt

Допустимый минимальный каркас сигнатур:

```
public class PortalOptions
{
    public string Title { get; set; } = "";
    public string Semester { get; set; } = "";
    public string SupportEmail { get; set; } = "";
    public AdminOptions Admin { get; set; } = new();
    public List<string> Modules { get; set; } = new();
}

public class NotificationOptions
{
    public string Sender { get; set; } = "";
    public string Channel { get; set; } = "";
    public string Signature { get; set; } = "";
}

public class TextConfigurationProvider : ConfigurationProvider
{
    public override void Load() { ... }
}

public class TextConfigurationSource : IConfigurationSource
{
    public IConfigurationProvider Build(IConfigurationBuilder builder) { ... }
}

public static class TextConfigurationExtensions
{
    public static IConfigurationBuilder AddTextFile(
        this IConfigurationBuilder builder, string path) { ... }
}

public class PortalHeaderMiddleware
{
    public PortalHeaderMiddleware(
        RequestDelegate next, IOptions<PortalOptions> options) { ... }

    public Task InvokeAsync(HttpContext context) { ... }
}
```

8. Требования к порядку источников и демонстрации приоритета

- Студент должен осознанно показать, что конфигурация собирается из нескольких источников, а не хранится в одном json-файле.
- Нужно продемонстрировать минимум два случая конфликта одинаковых ключей в разных источниках.
- В отчете обязательно должно быть объяснение, какое значение победило и за счет какого источника или порядка подключения.
- Маршрут `/config/providers` должен выводить реальные провайдеры, а не вручную написанный список строк.
- Маршрут `/config/effective` должен показывать итоговые значения конфликтующих ключей, а не только их базовые значения из одного файла.
- Для `portal.xml`, `notifications.ini` и `customsettings.txt` должно быть настроено копирование в выходной каталог приложения.

Важно: запрещено имитировать работу конфигурации вручную. Нельзя хардкодить ожидаемые значения в endpoint или подменять провайдер обычным `File.ReadAllText()` внутри маршрута. Преподаватель проверяет именно использование `IConfiguration` и реальных провайдеров.

9. Что должно войти в отчет студента

- скриншоты или список успешных запросов ко всем обязательным маршрутам;
- фрагмент `Program.cs` с порядком подключения источников конфигурации;
- фрагмент `launchSettings.json` с `environmentVariables` и `commandLineArgs`;
- скриншот ответа `/config/providers`;
- скриншот ответа `/config/effective` и пояснение, какой источник победил по каждому конфликтующему ключу;
- объяснение разницы между `Bind()/Get<T>()` и `IOptions<T>`;
- краткое описание формата `customsettings.txt` и алгоритма работы собственного провайдера;
- скриншот или доказательство наличия заголовков `X-Portal-Title` и `X-Portal-Semester` в ответе;

10. Критерии оценивания

Критерий	Баллы	Что проверяет преподаватель
Проект запускается и содержит все обязательные маршруты	10	Приложение работоспособно, маршруты возвращают осмысленные ответы.
Файловые источники JSON / XML / INI подключены корректно	10	Настройки реально читаются из файлов, а не подставляются вручную.

Нефайловые источники: command line, environment variables и in-memory	10	Есть минимум по одному примеру каждого типа источника.
Объединение конфигураций и демонстрация приоритета источников	15	При конфликте одинаковых ключей студент правильно объясняет итоговое значение.
Анализ конфигурации: GetSection, GetChildren, GetConnectionString, Providers	15	Маршруты /config/section/portal, /config/tree, /config/connection, /config/providers работают корректно.
Собственный провайдер конфигурации реализован корректно	10	Есть provider, source, extension method и чтение данных из customsettings.txt.
Проекция конфигурации на классы	10	Bind()/Get<T>() применены к сложному объекту с вложенностью и коллекцией.
IOptions и использование конфигурации в middleware	10	Настройки получаются через IOptions<T> и влияют на поведение middleware.
Отчет и объяснение логики конфигурации	10	Студент объясняет порядок источников, конфликты ключей и различие подходов.

11. Частые причины снятия баллов

- нет реального конфликта ключей между источниками, поэтому студент не может показать приоритет конфигурации;
- launchSettings.json не содержит commandLineArgs и дополнительных environmentVariables;
- маршрут /config/providers выводит статический текст вместо реального списка IConfigurationRoot.Providers;
- customsettings.txt читается напрямую в endpoint, а не через собственный IConfigurationProvider;
- Bind()/Get<T>() или IOptions<T> применены только к плоскому объекту без вложенности и без коллекции;
- middleware не использует IOptions<T> и не добавляет требуемые заголовки в ответ.

12. Дополнительные задания для сильных студентов

- сделать маршрут /config/tree с параметром ?section=..., чтобы анализировать любую секцию конфигурации;
- добавить HTML-таблицу на /config/providers с колонками Order, ProviderType и Notes;
- использовать Configure<PortalOptions>(opt => ...), чтобы программно переопределить одно из значений после чтения из файлов;
- связать через IOptions отдельную вложенную секцию, например Portal:Admin или Notifications;

- вывести на `/config/effective` не только итоговые значения, но и краткий комментарий о том, какой источник их переопределил.

Итог для студента

Сдать нужно не просто набор файлов, а понятную демонстрацию того, как ASP.NET Core собирает конфигурацию из разных источников. В защите студент должен уверенно объяснить разницу между `IConfiguration`, `Bind()/Get<T>()` и `IOptions<T>`, показать приоритет источников, а также обосновать, зачем вообще нужен собственный провайдер конфигурации.